PARAMFIT

User Manual

Generalized Born Solvation Parameter Fitting
with Fixed CDS Contributions

Ugur Bozkaya and Betul Ermis

May 15, 2026

Contents

1 Overview **1**

2 The [files] Block **1**
2.1 Recommended Minimal Usage . . . 1
2.2 Complete Usage . . . 2
2.3 Default Values . . . 2
2.4 ref_file . . . 3
2.5 output_file . . . 3
2.6 parameter_output_file . . . 4

3 The [data] Block **5**
3.1 Recommended Minimal Usage . . . 5
3.2 Complete Usage . . . 5
3.3 Default Values . . . 5
3.4 ndata . . . 6
3.5 qcmethod . . . 6
3.6 charge_type . . . 6
3.7 MacroQC Output Requirements . . . 7
3.8 Important Practical Notes . . . 8

4 The [parameters] Block **8**
4.1 Recommended Minimal CHO Example . . . 8
4.2 Complete Usage . . . 9
4.3 Default Values . . . 9
4.4 initial_parameter_file . . . 10
4.5 General Model Constants . . . 10
4.5.1 solvent_eps . . . 10
4.5.2 gb_sij . . . 10
4.5.3 gb_cij . . . 11
4.6 GB Distance Parameters . . . 11
4.6.1 gb_dij . . . 11
4.6.2 dch, doh, and dnh . . . 11
4.7 Atom Selection . . . 11
4.7.1 fit_atoms . . . 11
4.7.2 base_atoms . . . 12
4.8 Sij Parametrization . . . 12
4.8.1 sij_mode . . . 12
4.8.2 sij_group_scheme . . . 12
4.8.3 sij_pair_selection . . . 12
4.9 Optimization Flags . . . 13
4.10 Typical Stage Examples . . . 13

5 The [optimization] Block **14**
5.1 Recommended Global Usage . . . 14
5.2 Complete Usage . . . 14
5.3 Default Values . . . 14

5.4 `algorithm` . . . 15
5.5 `objective` . . . 15
5.6 `max_penalty_weight` . . . 16
5.7 Global Optimizer Options . . . 16
5.7.1 `global_maxiter` . . . 16
5.7.2 `popsize` . . . 16
5.7.3 `seed` . . . 16
5.7.4 `polish` . . . 17
5.8 Local Optimizer Options . . . 17
5.8.1 `max_nfev` . . . 17
5.8.2 `ftol`, `xtol`, and `gtol` . . . 17
5.9 Choosing Between Global and Local Optimization . . . 17
5.10 Typical Examples . . . 18

6 The `[tolerances]` Block **18**
6.1 Recommended Usage . . . 18
6.2 Default Values . . . 19
6.3 `mae_tol` . . . 19
6.4 `rms_tol` . . . 19
6.5 `max_tol` . . . 19
6.6 How to Interpret the Tolerance Summary . . . 19

7 Running PARAMFIT **20**
7.1 Editable Installation . . . 20
7.2 Recommended Command . . . 20
7.3 Alternative Python Module Command . . . 20
7.4 Working Directory Requirements . . . 21

8 Input File Examples **21**
8.1 Stage 1: CHO Fit . . . 21
8.2 Later Stage With Frozen CHO Parameters . . . 22
8.3 Manual Explicit Sij Pair Selection . . . 22

9 Generated Output Files **23**
9.1 Main Output File . . . 23
9.2 Parameter Output File . . . 23
9.3 Parameter File Contents . . . 24

10 Recommended Staged Workflow **24**
10.1 General Rule . . . 24
10.2 Suggested Stage Order . . . 25
10.3 Example Stage Logic . . . 25

11 Troubleshooting and Common Mistakes **25**
11.1 The Fit Starts From Default Parameters . . . 25
11.2 The Optimizer Does Not Meet the Tolerances . . . 25
11.3 A Parameter Hits Its Bound . . . 25
11.4 Sij Pairs Are Missing . . . 26
11.5 MAE Is Acceptable but Maximum Error Is Large . . . 26

11.6 Global Optimization Is Slow 26

1 Overview

Paramfit is a command-line program for fitting parameters in a Generalized Born / pairwise descreening solvation model against reference solvation free energies. The program reads molecular data from previously computed output files, keeps the CDS contribution fixed, recomputes the polarization contribution with trial parameter sets, and optimizes selected model parameters against the supplied reference data.

The recommended way to run the program is to provide a TOML-style input file:

```
paramfit --input job_name.inp
```

The input file is divided into named blocks. Each block controls a different part of the job:

```
[files]
[data]
[parameters]
[optimization]
[tolerances]
```

This manual describes each block in detail. The first block, `[files]`, controls the names of the reference data file, the main output file, and the parameter file written at the end of the fit.

2 The `[files]` Block

The `[files]` block defines the file names used by a Paramfit job. It is intentionally small: it does not control the molecular data format, the fitting model, or the optimization algorithm. Its only purpose is to tell the program where to read the reference values from and where to write the job outputs.

All options in this block have default values. Therefore, a valid input file may omit one or more entries from the `[files]` block. If an entry is omitted, Paramfit uses the corresponding default described below. In routine work, it is often best to provide only the reference file name and allow the program to derive the output file names from the input file name.

2.1 Recommended Minimal Usage

For most jobs, the recommended `[files]` block is:

```
[files]
ref_file = "ref.txt"
```

With this minimal form, Paramfit reads reference solvation free energies from `ref.txt`. The main output file and the final parameter file are then generated automatically from the input file name.

For example, if the job is launched as:

```
paramfit --input cho_stage.inp
```

and no explicit output file names are given, the program will use:

File role	File name
Input file	`cho_stage.inp`
Main output file	`cho_stage.out`
Final parameter file	`cho_stage.params.toml`

This naming convention is recommended because it keeps the input, output, and final parameter file associated with the same job name.

2.2 Complete Usage

All available options in the `[files]` block are:

```
[files]
ref_file = "ref.txt"
output_file = "paramfit.out"
parameter_output_file = "paramfit.params.toml"
```

Each option is described below.

2.3 Default Values

If the user does not provide any custom file names, the following defaults are used:

Option	Default value	Meaning
`ref_file`	`ref.txt`	Reference solvation free energies
`output_file`	derived from input name	Main text output
`parameter_output_file`	derived from output name	Final parameter file

The output-name default has two cases:

- If the program is run with an input file, the main output file is derived from the input file name. For example, `cho.inp` gives `cho.out`.
- If the program is run without an input file, the main output file is `paramfit.out`.

The parameter file is derived from the final main output file name. For example, `cho.out` gives `cho.params.toml`, and `paramfit.out` gives `paramfit.params.toml`.

Thus, the following minimal block:

```
[files]
```

is valid and means:

```
[files]
ref_file = "ref.txt"
# output_file is derived automatically
# parameter_output_file is derived automatically
```

However, it is usually clearer to write at least the reference file explicitly:

```
[files]
ref_file = "ref.txt"
```

2.4 `ref_file`

```
ref_file = "ref.txt"
```

Purpose. Specifies the file containing the reference solvation free energies used as target values during fitting.

Default. If `ref_file` is not provided, the program uses:

```
ref_file = "ref.txt"
```

Expected format. The reference file must contain one data point per line. Each line must contain the data index followed by the reference solvation free energy in kcal/mol:

```
1  -6.31
2  -4.29
3  -0.87
4   1.83
```

The first column is the data index. The second column is the reference solvation free energy. The order and number of entries must be consistent with the calculation output files in the working directory.

Index matching. PARAMFIT expects calculation output files to follow the naming pattern:

```
[index]_[name].out
```

For example:

```
1_water.out
2_methanol.out
3_acetone.out
```

The index in the reference file must match the index at the beginning of the corresponding output file name. For example, line `2 -4.29` in the reference file corresponds to the output file whose name starts with `2_`.

Typical stage-specific names. For staged parametrization, it is often clearer to use stage-specific reference file names:

```
ref_file = "ref_cho.txt"
ref_file = "ref_chon.txt"
ref_file = "ref_chons.txt"
```

These names are not required by the program. They are only a recommended project organization convention.

2.5 `output_file`

```
output_file = "paramfit.out"
```

Purpose. Specifies the main text output file written by the program. This file contains the job summary, input summary, initial parameters, optimization progress, final parameters, final statistics, bound checks, and the full final error table.

Default behavior. The default depends on how the program is launched.

- If no input file is used, the default is `paramfit.out`.

- If an input file is used and `output_file` is not specified, the output file name is derived from the input file name.

  For example:

Input file	Default output file
`cho.inp`	`cho.out`
`n_stage.inp`	`n_stage.out`
`halogen_fit.inp`	`halogen_fit.out`

When to specify it manually. Manual specification is useful when several fits are launched from the same input file but should write to different output names:

```
[files]
ref_file = "ref_cho.txt"
output_file = "paramfit_cho_seed_1.out"
```

However, for routine usage, deriving the output name from the input file name is usually cleaner.

2.6 `parameter_output_file`

```
parameter_output_file = "paramfit.params.toml"
```

Purpose. Specifies the TOML parameter file written at the end of the optimization. This file contains the optimized radii, Sij values, and GB distance parameters. It is the file that should normally be passed to the next stage through `initial_parameter_file` in the `[parameters]` block.

Default behavior. If `parameter_output_file` is not provided, the program derives it from the main output file name by replacing the suffix with `.params.toml`.

Main output file	Default parameter file
`paramfit.out`	`paramfit.params.toml`
`cho_stage.out`	`cho_stage.params.toml`
`n_stage.out`	`n_stage.params.toml`

Use in staged parametrization. The parameter file written by one stage can be used as the initial parameter file for the next stage. For example, a CHO fit may write:

```
parameter_output_file = "cho_stage.params.toml"
```

The next stage may then read it as:

```
[parameters]
initial_parameter_file = "../1_CHO/cho_stage.params.toml"
```

This mechanism is central to staged parametrization: previously optimized parameters are loaded and kept fixed unless the new input explicitly requests that they be optimized again.

3 The [data] Block

The [data] block defines how Paramfit interprets the calculation output files in the current working directory. It does not contain the molecular data directly. Instead, it tells the program how many data points to read, which quantum chemistry method label to search for, and which atomic charge block should be used in the polarization energy calculation.

Paramfit currently reads MacroQC output files only. The expected files must be plain-text .out files in the working directory, and their names must follow the index-based convention:

```
[index]_[name].out
```

Examples are:

```
1_water.out
2_methanol.out
3_acetone.out
```

The leading integer is important. It connects the calculation output file to the corresponding line in the reference file. For example, the output file 3_acetone.out corresponds to the reference entry whose first column is 3.

3.1 Recommended Minimal Usage

For the current SCF/IAO-Mulliken workflow, the [data] block can often be omitted or kept minimal:

```
[data]
qcmethod = "SCF"
charge_type = "IAO_MULLIKEN"
```

If ndata is not supplied, Paramfit counts the number of reference entries and the number of output files automatically. This is the recommended behavior for routine use because it catches missing or extra output files early.

3.2 Complete Usage

All available options in the [data] block are:

```
[data]
ndata = 115
qcmethod = "SCF"
charge_type = "IAO_MULLIKEN"
```

3.3 Default Values

If the user does not provide custom values, the following defaults are used:

Option	Default value	Meaning
ndata	automatically counted	Number of data points
qcmethod	SCF	QC method label used in MacroQC output parsing
charge_type	IAO_MULLIKEN	Atomic charge block used for GB polarization

3.4 ndata

```
ndata = 115
```

Purpose. Specifies the number of data points in the fitting set.

Default behavior. If `ndata` is not provided, PARAMFIT counts:

- the number of valid entries in the reference file, and
- the number of calculation output files matching `[0-9]*_*.out`.

The two counts must agree. If they do not agree, the program stops with an error unless the user explicitly provides `ndata`. This check is intended to avoid accidental fits with missing output files, extra files in the directory, or reference entries that do not have corresponding calculations.

When to set it manually. In most production runs, `ndata` should be omitted. It should be set manually only when the user intentionally wants to override the automatic counting behavior.

3.5 qcmethod

```
qcmethod = "SCF"
```

Purpose. Specifies the quantum chemistry method label used when searching MacroQC output files. This string is not used to perform a quantum chemistry calculation. It is only used to locate method-specific blocks and energy lines that already exist in the output files.

Default. If `qcmethod` is not provided, the program uses:

```
qcmethod = "SCF"
```

How it is used. The value of `qcmethod` is inserted into the text patterns searched by the parser. For example, with:

```
qcmethod = "SCF"
```

the program searches for charge and CDS labels such as:

```
Mulliken Population Analysis for SCF-IAO:
Lowdin Population Analysis for SCF-IAO:
SCF::CDS Free Energy (kcal/mol):
```

If a different method label is used in the MacroQC output, `qcmethod` must match that label. For example, if the output contains method-specific lines for `MP2`, the input should use:

```
qcmethod = "MP2"
```

The label must be consistent with the text printed in the output files. If the corresponding charge block or CDS free energy line cannot be found, the program stops with a parsing error.

3.6 charge_type

```
charge_type = "IAO_MULLIKEN"
```

Purpose. Specifies which atomic charge block is read from the MacroQC output files and used in the GB polarization energy calculation.

Default. If `charge_type` is not provided, the program uses:

```
charge_type = "IAO_MULLIKEN"
```

Supported behavior. The current parser distinguishes two charge-block styles:

- `IAO_MULLIKEN`: reads the block labeled `Mulliken Population Analysis for <qcmethod>-IAO:`
- any other value: reads the block labeled `Lowdin Population Analysis for <qcmethod>-IAO:`

Therefore, the recommended explicit values are:

```
charge_type = "IAO_MULLIKEN"
charge_type = "IAO_LOWDIN"
```

The charge type affects only the atomic charges used in the polarization calculation. It does not change the CDS contribution. The CDS contribution is read separately from the method-specific CDS free energy line and is kept fixed during parameter fitting.

3.7 MacroQC Output Requirements

Each MacroQC output file must contain all information required to recompute the polarization energy and add the fixed CDS contribution. The current parser expects the following data to be present:

Required data	How it is identified
Number of atoms	`Number of Atoms : <integer>`
Cartesian coordinates	indexed lines containing atom symbol and x/y/z values
Atomic charges	selected Mulliken or Lowdin `<qcmethod>-IAO` block
CDS free energy	`<qcmethod>::CDS Free Energy (kcal/mol): <value>`

Number of atoms. The parser searches for a line of the form:

```
Number of Atoms : 12
```

Cartesian coordinates. The parser reads coordinate lines containing an atom index, an element symbol, and three Cartesian coordinates. A typical line has the form:

```
1  C   0.000000   0.000000   0.000000
```

The coordinates are read in Angstrom.

Atomic charges. The parser first locates the requested charge-analysis block, then reads floating-point charge values until the block ends. At least one charge per atom must be present.

CDS free energy. The parser searches for the method-specific CDS free energy line. For the default method, the expected line contains:

```
SCF::CDS Free Energy (kcal/mol):
```

The numerical value on this line is read in kcal/mol and kept fixed throughout the optimization.

3.8 Important Practical Notes

- Paramfit does not run MacroQC. It only reads existing MacroQC output files.
- All output files for the current fit must be in the directory where the program is launched.
- Output files must follow the `[index]_[name].out` naming rule.
- The reference file indices and output file indices must describe the same molecules in the same dataset.
- The selected `qcmethod` and `charge_type` must match labels that actually exist in the output files.
- The CDS energy is not fitted. It is read from each output file and added as a fixed contribution to the recomputed polarization energy.

4 The [parameters] Block

The `[parameters]` block controls the solvation-model parameters used by the fit. It specifies the starting values for model constants, the atom types to be optimized, the Sij parametrization mode, and the nonlinear GB distance parameters that should be optimized.

This block is the central part of a Paramfit input file. The reference energies and molecular data are read elsewhere, but the `[parameters]` block determines which physical parameters are allowed to change during the optimization.

4.1 Recommended Minimal CHO Example

For a first-stage CHO fit using grouped Sij parameters and optimized C–H and O–H distance parameters, a typical `[parameters]` block is:

```
[parameters]
solvent_eps = 78.3553
gb_sij = 0.75
gb_dij = 3.7
dch = 3.7
doh = 3.7
gb_cij = 0.0

fit_atoms = ["H", "C", "O"]
sij_mode = "grouped"
sij_group_scheme = "hco"

optimize_sij = true
optimize_dij = true
optimize_dch = true
optimize_doh = true
```

For later stages, an initial parameter file is normally provided so that previously optimized parameters can be reused:

```
[parameters]
initial_parameter_file = "../1_CHO/cho_stage.params.toml"
```

```
fit_atoms = ["N"]
base_atoms = ["H", "C", "O"]

sij_mode = "explicit"
sij_pair_selection = "fit-with-base"

optimize_sij = true
optimize_dnh = true
```

4.2 Complete Usage

All available options in the `[parameters]` block are:

```
[parameters]
initial_parameter_file = "previous_stage.params.toml"

solvent_eps = 78.3553
gb_sij = 0.75
gb_dij = 3.7
dch = 3.7
doh = 3.7
dnh = 3.7
gb_cij = 0.0

fit_atoms = ["H", "C", "O"]
base_atoms = ["H", "C", "O"]

sij_mode = "explicit"
sij_group_scheme = "hco"
sij_pair_selection = "auto"
fit_sij_pairs = [["H", "C"], ["C", "H"]]

optimize_sij = false
optimize_dij = false
optimize_dch = false
optimize_doh = false
optimize_dnh = false
```

4.3 Default Values

If the user does not provide custom values, the following defaults are used:

Option	Default value	Meaning
`initial_parameter_file`	not provided	start from built-in defaults
`solvent_eps`	`78.3553`	solvent dielectric constant
`gb_sij`	`0.75`	default Sij value
`gb_dij`	`3.7`	default GB distance parameter, d0
`dch`	`3.7`	C–H / H–C distance parameter
`doh`	`3.7`	O–H / H–O distance parameter
`dnh`	`3.7`	N–H / H–N distance parameter
`gb_cij`	`0.0`	additive GB gamma parameter
`fit_atoms`	`["H", "C", "O"]`	atom types whose rho values are fit
`base_atoms`	empty list	atom types from previous stages
`sij_mode`	`explicit`	explicit Sij fitting mode
`sij_pair_selection`	`auto`	automatic explicit pair generation
`optimize_*`	`false`	parameter is fixed unless enabled

4.4 `initial_parameter_file`

```
initial_parameter_file = "../1_CHO/cho_stage.params.toml"
```

Purpose. Reads a parameter file generated by a previous fit. This is the standard way to perform staged parametrization. For example, CHO parameters can be optimized in the first stage, written to a parameter file, and then loaded in the N stage.

Default behavior. If `initial_parameter_file` is not provided, no previous parameter file is read. The program starts from built-in default values for rho, Sij, and GB distance parameters. The output file prints a warning explaining that no initial parameter file was supplied.

Use in staged fitting. When this option is provided, previously optimized values are loaded before the new optimization starts. Loaded parameters remain fixed unless the current input explicitly requests that they be optimized again.

4.5 General Model Constants

4.5.1 `solvent_eps`

```
solvent_eps = 78.3553
```

Solvent dielectric constant used in the GB polarization energy. The default value, `78.3553`, is appropriate for water at standard conditions used in the current workflow.

4.5.2 `gb_sij`

```
gb_sij = 0.75
```

Default pairwise descreening scale factor. If Sij parameters are not optimized, this value is used as the global Sij value. If Sij parameters are optimized, the initial values normally come from the internal Sij table or from a loaded parameter file, but `gb_sij` remains the global fallback value.

4.5.3 gb_cij

```
gb_cij = 0.0
```

Additive term in the GB gamma expression. The default is zero. In the current parametrization workflow, this parameter is normally kept fixed.

4.6 GB Distance Parameters

The GB distance parameters control the denominator used in the off-diagonal GB gamma terms. Paramfit supports one general distance parameter and three special hydrogen-containing pair parameters:

Parameter	Applied atom pairs
`gb_dij`	all pairs not covered by `dch`, `doh`, or `dnh`
`dch`	C–H and H–C
`doh`	O–H and H–O
`dnh`	N–H and H–N

The pair matching is symmetric. Therefore, `dch` is used for both C–H and H–C, `doh` is used for both O–H and H–O, and `dnh` is used for both N–H and H–N.

4.6.1 gb_dij

```
gb_dij = 3.7
```

General GB distance parameter, also referred to as d0. It is used for every atom pair that is not C–H, O–H, or N–H. Examples include C–C, C–O, O–O, H–H, C–N, S–H, P–H, and halogen–H pairs.

4.6.2 dch, doh, and dnh

```
dch = 3.7
doh = 3.7
dnh = 3.7
```

Special GB distance parameters for C–H, O–H, and N–H pairs. These values may be fixed or optimized depending on the corresponding optimization flags.

4.7 Atom Selection

4.7.1 fit_atoms

```
fit_atoms = ["H", "C", "O"]
```

Atom types whose intrinsic Coulomb radii, rho, are included in the optimization vector. If `fit_atoms = ["N"]`, only rho for N is optimized, while previously loaded H, C, and O rho values remain fixed.

4.7.2 base_atoms

```
base_atoms = ["H", "C", "O"]
```

Atom types that have already been parametrized in previous stages. This option is mainly used with `sij_pair_selection = "fit-with-base"` to generate Sij pairs between newly fitted atom types and previously fitted atom types.

4.8 Sij Parametrization

4.8.1 sij_mode

```
sij_mode = "explicit"
```

Fits selected Sij pairs independently. This is useful for later stages where a new atom type is added and only pairs involving the new atom should be fitted.

```
sij_mode = "grouped"
```

Fits Sij values through a predefined grouping scheme. Grouped fitting reduces the number of independent Sij parameters and is useful when a stage should remain close to a compact parametrization model.

4.8.2 sij_group_scheme

```
sij_group_scheme = "hco"
```

Uses the grouped CHO Sij scheme. This option is meaningful only when `sij_mode = "grouped"`.

```
sij_group_scheme = "halogen"
```

Uses the grouped halogen Sij scheme. In this scheme, halogens are treated as a shared group for selected Sij patterns.

4.8.3 sij_pair_selection

This option is used only when `sij_mode = "explicit"`.

```
sij_pair_selection = "auto"
```

Generates all ordered pairs within `fit_atoms`. For `fit_atoms = ["H", "C", "O"]`, this produces H–H, H–C, H–O, C–H, C–C, C–O, O–H, O–C, and O–O.

```
sij_pair_selection = "fit-with-base"
```

Generates Sij pairs between `fit_atoms` and `base_atoms`, in both directions, and also includes the self-pair of each fitted atom. For example:

```
fit_atoms = ["N"]
base_atoms = ["H", "C", "O"]
sij_pair_selection = "fit-with-base"
```

generates N–N, N–H, H–N, N–C, C–N, N–O, and O–N.

```
sij_pair_selection = "manual"
fit_sij_pairs = [["N", "H"], ["H", "N"]]
```

Uses exactly the Sij pairs listed in `fit_sij_pairs`.

4.9 Optimization Flags

The following Boolean flags determine which parameters are included in the optimizer vector:

Flag	Effect
`optimize_sij`	optimize selected Sij parameters
`optimize_dij`	optimize `gb_dij`, the general d0 parameter
`optimize_dch`	optimize the C–H / H–C distance parameter
`optimize_doh`	optimize the O–H / H–O distance parameter
`optimize_dnh`	optimize the N–H / H–N distance parameter

If a flag is `false`, the corresponding parameter is fixed at its input value or at the value loaded from `initial_parameter_file`. If a flag is `true`, the corresponding parameter is optimized starting from that value.

4.10 Typical Stage Examples

CHO stage.

```
[parameters]
solvent_eps = 78.3553
gb_sij = 0.75
gb_dij = 3.7
dch = 3.7
doh = 3.7
gb_cij = 0.0

fit_atoms = ["H", "C", "O"]
sij_mode = "grouped"
sij_group_scheme = "hco"

optimize_sij = true
optimize_dij = true
optimize_dch = true
optimize_doh = true
```

N stage.

```
[parameters]
initial_parameter_file = "../1_CHO/cho_stage.params.toml"

fit_atoms = ["N"]
base_atoms = ["H", "C", "O"]

sij_mode = "explicit"
sij_pair_selection = "fit-with-base"

optimize_sij = true
optimize_dnh = true
```

5 The [optimization] Block

The [optimization] block controls the numerical optimization procedure. It selects the optimizer, the scalar objective function used by the global optimizer, population-search settings, local least-squares settings, and convergence tolerances passed to the local optimizer.

This block does not choose which physical parameters are optimized. That choice is made in the [parameters] block through flags such as optimize_sij, optimize_dij, optimize_dch, optimize_doh, and optimize_dnh. The [optimization] block controls how the selected parameter vector is optimized.

5.1 Recommended Global Usage

For production parametrization, the recommended starting point is a global differential-evolution search using the MAE/MAX objective:

```
[optimization]
algorithm = "global"
objective = "mae-max"
max_penalty_weight = 1.0

global_maxiter = 80
popsize = 15
seed = 1
polish = true
```

This setup performs a population-based global search and then optionally applies a local polishing step. It is usually more appropriate than a purely local least-squares fit when the optimized parameters are nonlinear and bounded.

5.2 Complete Usage

All available options in the [optimization] block are:

```
[optimization]
algorithm = "global"
objective = "mae-max"
max_penalty_weight = 1.0

global_maxiter = 80
popsize = 15
seed = 1
polish = true

max_nfev = 200
ftol = 1.0e-12
xtol = 1.0e-12
gtol = 1.0e-12
```

5.3 Default Values

If the user does not provide custom values, the following defaults are used:

Option	Default value	Meaning
`algorithm`	local	optimizer selection
`objective`	`mae`	scalar objective for global search
`max_penalty_weight`	`1.0`	penalty weight for `mae-max`
`global_maxiter`	`80`	differential-evolution generations
`popsize`	`15`	population multiplier
`seed`	`1`	random seed for global search
`polish`	`true`	final local refinement after global search
`max_nfev`	`200`	maximum local function evaluations
`ftol`	`1.0e-12`	local optimizer cost tolerance
`xtol`	`1.0e-12`	local optimizer step tolerance
`gtol`	`1.0e-12`	local optimizer gradient tolerance

5.4 `algorithm`

```
algorithm = "global"
```

Selects the global optimizer. In this mode, Paramfit uses SciPy's `differential_evolution` routine. This is a population-based optimizer. It explores the bounded parameter space by maintaining a population of trial parameter vectors and evolving them over multiple generations.

```
algorithm = "local"
```

Selects the local optimizer. In this mode, Paramfit uses `scipy.optimize.least_squares`. The local optimizer minimizes the vector of signed residuals:

```
residual_i = calculated_i - reference_i
```

The local optimizer can be useful when the starting parameters are already close to a good solution. It is less robust when the initial guess is poor or when the objective landscape has multiple local minima.

Default behavior. If `algorithm` is not provided, the program uses the local optimizer. For nonlinear parametrization work, explicitly setting `algorithm = "global"` is usually recommended.

5.5 `objective`

```
objective = "mae"
```

Minimizes the mean absolute error:

```
MAE = mean(abs(calculated_i - reference_i))
```

```
objective = "rmse"
```

Minimizes the root mean square error:

```
RMSE = sqrt(mean((calculated_i - reference_i)^2))
```

```
objective = "mae-max"
```

Minimizes MAE while penalizing large maximum errors. The objective is:

```
objective = MAE + max_penalty_weight * max(0, MAX - max_tol)^2
```

Here `MAX` is the largest absolute error in the data set. The `max_tol` value is defined in the `[tolerances]` block.

Important. The `objective` option is used by the global optimizer. The local least-squares optimizer always works with the signed residual vector, because that is the mathematical interface required by `least_squares`.

5.6 max_penalty_weight

```
max_penalty_weight = 1.0
```

Controls the strength of the maximum-error penalty when `objective = "mae-max"`. Increasing this value makes the optimizer more aggressive about reducing the worst error once it exceeds `max_tol`. Decreasing it makes the objective behave more like pure MAE.

This option has no effect when `objective = "mae"` or `objective = "rmse"`.

5.7 Global Optimizer Options

The following options are used when:

```
algorithm = "global"
```

5.7.1 global_maxiter

```
global_maxiter = 80
```

Maximum number of differential-evolution generations. Larger values allow a longer global search and may improve the result, but the calculation becomes more expensive.

5.7.2 popsize

```
popsize = 15
```

Population-size multiplier used by differential evolution. The approximate number of parameter vectors in the population is:

```
population size = popsize * number_of_optimized_parameters
```

For example, if 12 parameters are optimized and `popsize = 15`, the population contains approximately 180 trial parameter vectors. Increasing `popsize` improves sampling of the parameter space but increases the number of energy evaluations per generation.

5.7.3 seed

```
seed = 1
```

Random seed for the global optimizer. Using a fixed seed makes the run reproducible. Changing the seed is a useful way to test whether the result is robust or whether the optimizer is finding different local regions of parameter space.

5.7.4 polish

```
polish = true
```

If `true`, SciPy applies a final local minimization step after the differential-evolution search. This can improve the final objective value. For nonsmooth objectives such as `mae-max`, polishing may take additional time and may not always improve the result substantially.

If `false`, the final result is the best parameter vector found by differential evolution itself.

5.8 Local Optimizer Options

The following options are used when:

```
algorithm = "local"
```

They are also relevant indirectly when global polishing is enabled, although the main local least-squares settings are most important for explicit local runs.

5.8.1 max_nfev

```
max_nfev = 200
```

Maximum number of function evaluations allowed for the local least-squares optimizer. If the value is too small, the optimizer may stop before fully converging. If it is too large, difficult fits may spend excessive time in local optimization.

5.8.2 ftol, xtol, and gtol

```
ftol = 1.0e-12
xtol = 1.0e-12
gtol = 1.0e-12
```

Convergence tolerances passed to `scipy.optimize.least_squares`.

- `ftol`: tolerance for changes in the cost function.
- `xtol`: tolerance for changes in the parameter vector.
- `gtol`: tolerance for the gradient-based optimality condition.

Small values request a tight local convergence. They do not guarantee that the fit will satisfy the MAE, RMSE, or MAX targets printed in the output; those targets are model-quality thresholds, not optimizer stopping criteria.

5.9 Choosing Between Global and Local Optimization

Use global optimization when fitting nonlinear parameters such as rho, Sij, `gb_dij`, `dch`, `doh`, or `dnh`, especially when starting from default parameters or when the fit may have multiple local minima.

Use local optimization when starting from a reliable parameter file and only making a small refinement. Local optimization is faster, but it depends more strongly on the starting point.

5.10 Typical Examples

Global MAE/MAX fit.

```
[optimization]
algorithm = "global"
objective = "mae-max"
max_penalty_weight = 1.0
global_maxiter = 80
popsize = 15
seed = 1
polish = true
```

Pure MAE global fit.

```
[optimization]
algorithm = "global"
objective = "mae"
global_maxiter = 80
popsize = 15
seed = 1
polish = true
```

Local least-squares refinement.

```
[optimization]
algorithm = "local"
max_nfev = 500
ftol = 1.0e-12
xtol = 1.0e-12
gtol = 1.0e-12
```

6 The [tolerances] Block

The [tolerances] block defines the error thresholds used when PARAMFIT reports whether a fitted parameter set satisfies the requested quality targets. These values are printed in the output file and are used in the final pass/fail summary.

Important distinction. The tolerances in this block are not the mathematical stopping criteria of the optimizer. They do not force the optimizer to stop exactly when all targets are met, and they do not guarantee that the final solution will satisfy every target. They are model-quality thresholds used to interpret the final errors. The optimizer stopping criteria are controlled separately in the [optimization] block.

6.1 Recommended Usage

```
[tolerances]
mae_tol = 0.50
rms_tol = 0.80
max_tol = 2.50
```

These values are strict enough to be useful for monitoring the fit, while still being realistic for early-stage parametrization. For a publication-quality parametrization, the final acceptable

thresholds should be chosen based on the training set, the chemical domain, and comparison with the reference method.

6.2 Default Values

Option	Default	Meaning
`mae_tol`	`0.50`	Target mean absolute error in kcal/mol
`rms_tol`	`0.80`	Target root mean square error in kcal/mol
`max_tol`	`2.50`	Target maximum absolute error in kcal/mol

6.3 `mae_tol`

```
mae_tol = 0.50
```

Defines the desired upper limit for the mean absolute error:

```
MAE = mean(abs(calculated_i - reference_i))
```

MAE is often the primary metric for solvation-energy parametrization because it measures the typical unsigned error in the fitted data set. A low MAE means that the fitted model performs well on average, but it does not guarantee that every individual molecule is well described.

6.4 `rms_tol`

```
rms_tol = 0.80
```

Defines the desired upper limit for the root mean square error:

```
RMSE = sqrt(mean((calculated_i - reference_i)^2))
```

RMSE gives larger weight to molecules with large errors. If RMSE is much larger than MAE, the data set likely contains one or more difficult cases or outliers. This does not automatically mean the fit is wrong, but it should be inspected.

6.5 `max_tol`

```
max_tol = 2.50
```

Defines the desired upper limit for the largest absolute error in the data set:

```
MAX = max(abs(calculated_i - reference_i))
```

The maximum error is useful for identifying the worst-described molecule. It is also used when `objective = "mae-max"` is selected in the `[optimization]` block. In that case, the global optimizer minimizes MAE while adding a penalty when `MAX` exceeds `max_tol`.

6.6 How to Interpret the Tolerance Summary

At the end of a run, Paramfit prints the final MAE, RMSE, and maximum absolute error together with the tolerance values. A typical interpretation is:

- If MAE, RMSE, and MAX all pass, the fit satisfies the requested error targets for the training set.
- If MAE passes but MAX fails, the average model is acceptable but one or a few molecules should be inspected.
- If MAE fails, the parameter model, training data, fixed parameters, or selected fitting stage should be reconsidered.
- If RMSE is much larger than MAE, the fit is probably dominated by a small number of difficult molecules.

7 Running PARAMFIT

The recommended way to run Paramfit is to place a complete input file in the working directory containing the reference file and MacroQC output files, and then run the program with the `-i` option.

7.1 Editable Installation

During development, install the package in editable mode from the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
```

Editable installation means that Python imports the source code directly from the repository. After this installation, normal changes to the Python source files do not require running `python -m pip install -e .` again. Reinstallation is only needed when the packaging metadata or installed entry points are changed.

7.2 Recommended Command

```
paramfit -i input.inp
```

This command reads the input file, loads the reference data and MacroQC output files, runs the requested optimization, writes the main output file, and writes a parameter file that can be used as input for later stages.

7.3 Alternative Python Module Command

If the console command is not available, the package can also be run as a Python module:

```
python -m paramfit.cli -i input.inp
```

This is equivalent to the `paramfit -i input.inp` command.

7.4 Working Directory Requirements

Paramfit expects to be run from the directory containing the data files for the current fitting stage. The directory should contain:

- the input file, for example `cho.inp`;
- the reference file, usually `ref.txt` unless another name is given;
- MacroQC output files named with the expected indexed pattern;
- any initial parameter file required by the current stage.

The package source directory does not have to be the same as the data directory. After editable installation, the command can be executed from any stage directory as long as the active Python environment can import the installed package.

8 Input File Examples

This section gives compact examples of complete input files. The numerical values shown here are starting values and workflow examples, not universal final parameters.

8.1 Stage 1: CHO Fit

```
[files]
ref_file = "ref.txt"
output_file = "cho_fit.out"
parameter_output_file = "cho_fit.params.toml"

[data]
qcmethod = "SCF"
charge_type = "IAO_MULLIKEN"

[parameters]
fit_atoms = ["H", "C", "O"]
solvent_eps = 78.3553
gb_sij = 0.75
gb_dij = 3.7
dch = 3.7
doh = 3.7
dnh = 3.7
gb_cij = 0.0
sij_mode = "grouped"
sij_group_scheme = "cho"
optimize_sij = true
optimize_dch = true
optimize_doh = true

[optimization]
algorithm = "global"
objective = "mae-max"
global_maxiter = 80
popsize = 15
```

```
seed = 1
polish = true

[tolerances]
mae_tol = 0.50
rms_tol = 0.80
max_tol = 2.50
```

8.2 Later Stage With Frozen CHO Parameters

```
[files]
ref_file = "ref.txt"
output_file = "chon_fit.out"
parameter_output_file = "chon_fit.params.toml"

[data]
qcmethod = "SCF"
charge_type = "IAO_MULLIKEN"

[parameters]
fit_atoms = ["N"]
base_atoms = ["H", "C", "O"]
initial_parameter_file = "../1_CHO/cho_fit.params.toml"
sij_mode = "explicit"
sij_pair_selection = "auto"
optimize_sij = true
optimize_dnh = true

[optimization]
algorithm = "global"
objective = "mae-max"
global_maxiter = 80
popsize = 15
seed = 1
polish = true

[tolerances]
mae_tol = 0.50
rms_tol = 0.80
max_tol = 2.50
```

In this example, the CHO parameters are read from the previous parameter file. Only the parameters selected by the current stage are optimized. Values loaded from `initial_parameter_file` are used as fixed starting parameters unless their corresponding optimization flag or Sij pair selection makes them active in the current fit.

8.3 Manual Explicit Sij Pair Selection

```
[parameters]
fit_atoms = ["N"]
base_atoms = ["H", "C", "O"]
initial_parameter_file = "../1_CHO/cho_fit.params.toml"
```

```
sij_mode = "explicit"
sij_pair_selection = "manual"
fit_sij_pairs = ["N_H", "N_C", "N_O", "N_N"]
optimize_sij = true
```

Manual pair selection should be used only when the user wants full control over the fitted Sij terms. For routine staged parametrization, `sij_pair_selection = "auto"` is safer because it avoids forgetting a pair that appears in the data set.

9 Generated Output Files

Paramfit writes two main files during a fitting run: the human-readable output file and the machine-readable parameter file.

9.1 Main Output File

The main output file is selected with:

```
[files]
output_file = "fit.out"
```

This file records the run setup, selected parameter model, optimized variables, final fitted values, and final error statistics. It is the primary file to read when judging whether a run is scientifically acceptable.

The output file is designed to answer the following questions:

- Which reference file and MacroQC output files were read?
- Which charge model and quantum-chemistry method were requested?
- Which atom types were active in the current fitting stage?
- Which rho, Sij, and distance parameters were optimized?
- Which parameters were held fixed?
- What are the final MAE, RMSE, and maximum absolute error?
- Which molecules have the largest residuals?

9.2 Parameter Output File

The parameter output file is selected with:

```
[files]
parameter_output_file = "fit.params.toml"
```

If `parameter_output_file` is not provided, Paramfit derives the name from the main output file by replacing the suffix with `.params.toml`. For example:

```
fit.out -> fit.params.toml
```

The parameter file is written in TOML format and can be used as `initial_parameter_file` in the next fitting stage.

9.3 Parameter File Contents

A generated parameter file contains metadata and fitted model parameters:

```
[metadata]
source_output = "fit.out"
sij_mode = "explicit"
objective = "mae-max"
cds_treatment = "fixed"

[rho_ang]
C = 1.2345678900
H = 1.2345678900
O = 1.2345678900

[sij]
C_H = 0.7500000000
C_O = 0.7500000000

[dij]
d0 = 3.7000000000
dch = 3.7000000000
doh = 3.7000000000
dnh = 3.7000000000
```

The `[rho_ang]` values are written in Angstrom. Internally, the fitting code converts radii as needed. The `[sij]` block stores pairwise Sij terms. The `[dij]` block stores the general distance parameter `d0` and the special hydrogen-pair parameters `dch`, `doh`, and `dnh`.

10 Recommended Staged Workflow

The intended use of Paramfit is staged parametrization. A small chemical subset is fitted first, then its optimized parameters are carried forward and held fixed while new atom types are introduced.

10.1 General Rule

For each stage:

1. Build a data directory containing only the molecules for that stage.
2. Use the previous stage's `.params.toml` file as `initial_parameter_file`.
3. Put newly introduced atom types in `fit_atoms`.
4. Put previously fitted atom types in `base_atoms`.
5. Optimize only the rho, Sij, and distance parameters that are chemically relevant to the newly introduced atom types.
6. Write a new `.params.toml` file for the next stage.

10.2 Suggested Stage Order

Stage	Chemical subset	Typical active atoms
1	CHO	`["H", "C", "O"]`
2	CHON	`["N"]` with CHO as base
3	CHONS	`["S"]` with CHON as base
4	CHONSP	`["P"]` with CHONS as base
5	Halogens	`["F", "CL", "BR"]` with CHONSP as base

This order keeps the parameter search smaller and makes the physical interpretation easier. It also reduces the risk that parameters for later atom types disturb the already optimized behavior of earlier subsets.

10.3 Example Stage Logic

For the CHO stage, optimize the CHO radii and the CHO-specific Sij and distance terms selected by the model. After the run, save the generated parameter file.

For the N stage, load the CHO parameter file, keep CHO parameters fixed, and fit only the N radius and N-containing Sij terms. If the model requires it, also fit `dnh`. The same pattern can be repeated for S, P, and halogens.

11 Troubleshooting and Common Mistakes

11.1 The Fit Starts From Default Parameters

If `initial_parameter_file` is not provided, Paramfit starts from the default parameter values given in the input file and code defaults. This is allowed, but it should be intentional. For later stages, forgetting `initial_parameter_file` usually means the fit is not using the parameters optimized in the earlier stage.

11.2 The Optimizer Does Not Meet the Tolerances

The tolerances are quality targets, not hard optimizer constraints. A run can finish normally while still failing one or more tolerances. If this happens, inspect:

- whether the correct reference file and MacroQC outputs were read;
- whether `fit_atoms` and `base_atoms` describe the intended stage;
- whether the correct initial parameter file was loaded;
- whether the selected parameter model is flexible enough for the data;
- whether a few molecules dominate the maximum error.

11.3 A Parameter Hits Its Bound

When a parameter converges to the lower or upper bound, the optimizer is asking for a value outside the allowed physical range. This can indicate that the current model is too restrictive, that the data contain difficult molecules, or that a previous-stage parameter should be revisited. It does not automatically mean the run failed, but it should be discussed and documented.

11.4 Sij Pairs Are Missing

For explicit Sij fitting, `sij_pair_selection = "auto"` is recommended unless there is a specific reason to use manual pair selection. Automatic selection examines the data and chooses the relevant pairs for the current `fit_atoms`. Manual selection is useful for controlled experiments, but it can silently omit an important pair if the list is incomplete.

11.5 MAE Is Acceptable but Maximum Error Is Large

This means the model is good on average but one or a few molecules are poorly described. The next step is to inspect the worst residuals in the output file. Possible explanations include an outlier molecule, an unusual functional group, an inconsistent reference value, or insufficient flexibility in the selected parameter model.

11.6 Global Optimization Is Slow

Global optimization cost increases with the number of fitted parameters, the population size, and the number of generations. The approximate population size is proportional to:

```
popsize * number_of_optimized_parameters
```

When many parameters are optimized simultaneously, increasing `global_maxiter` or `popsize` may improve the search but will also increase runtime. For large stages, it is often better to keep the parameter model chemically focused instead of fitting every possible parameter at once.